МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Выполнил: К. Смирнов
Группа: М8О-201БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков в управлении потоками в ОС и в обеспечении синхронизации между потоками.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Так же необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Вариант: 20. Дан массив координат (x, y, z). Необходимо найти три точки, которые образуют треугольник максимальной площади.

Метод решения

Площадь треугольника с вершинами A, B, C в пространстве считается через векторное произведение: $S = \frac{1}{2}|\vec{u} \times \vec{v}|$, где $\vec{u} = B - A$, $\vec{v} = C - A$. При переборе корень не берётся, сравнивается $|\vec{u} \times \vec{v}|^2$: функция $\sqrt{x}/2$ монотонная, поэтому максимум будет у той же тройки, а `sqrt` вызывается только один раз для ответа.

Перебираются все тройки индексов $i < j < k$, всего их $C_n^3 = \frac{n(n-1)(n-2)}{6}$, сложность $O(n^3)$. Условие $i < j < k$ нужно, чтобы не проверять один треугольник несколько раз и не брать тройки с повторяющейся точкой.

Для распараллеливания работа делится на «строки»: одна строка — это фиксированное i и все j, k после него. Строки независимы, но разные по объёму: в строке i примерно $(n-i)^2/2$ троек, то есть первые строки намного тяжелее последних. Если поделить строки на равные куски подряд, первый поток получит больше половины всей работы. Поэтому строки раздаются динамически: есть общий счётчик `next_row`, и каждый свободный поток под мьютексом берёт из него следующую строку. Так быстрые потоки берут больше строк, и все заканчивают примерно одновременно.

Каждый поток ищет максимум в своих строках в локальной переменной без блокировок. Когда строки кончаются, поток один раз под вторым мьютексом сравнивает свой результат с общим `best` и при необходимости обновляет его. Мьютексы захватываются только на выдаче строк (n раз) и на слиянии результата (t раз), поэтому потоки почти не мешают друг другу. Один поток никогда не держит два мьютекса сразу, поэтому взаимной блокировки быть не может. При равной площади побеждает тройка с меньшим i, чтобы ответ не зависел от количества потоков.

Количество потоков задаётся ключом `-t`: главный поток создаёт ровно t рабочих потоков через `pthread_create`, и других потоков во время работы не создаётся, поэтому одновременно работает не больше t потоков. Главный поток сам не считает, а ждёт рабочие потоки через `pthread_join`. Поэтому всего в процессе $t + 1$ поток.

Время измеряется через `clock_gettime(CLOCK_MONOTONIC)` только для поиска (создание потоков, перебор, join), генерация точек в замер не входит.

Описание программы

Проект собирается через CMake (по умолчанию в режиме Release) и разделён на файлы:

- include/points.h, src/points.c — структура Point с полями x, y, z и функции:
  - generate_points(n, seed) — генерирует n случайных точек с координатами в $[-1000, 1000]$;
  - read_points(filename, &n) — читает точки из файла (в первой строке количество, дальше по три числа в строке);
  - triangle_area(a, b, c) — площадь треугольника.
- include/search.h, src/search.c — многопоточный поиск:
  - структура Triangle — индексы i, j, k и значение $|\vec{u} \times \vec{v}|^2$;
  - структура ThreadArg — номер потока и сколько строк он посчитал;
  - worker — функция потока: берёт строки из счётчика, перебирает тройки, сливает результат;
  - lock, unlock — захват и освобождение мьютекса с проверкой кода возврата;
  - find_max_triangle(points, n, threads, &result) — создаёт мьютексы и потоки, ждёт их и освобождает ресурсы.
- src/main.c — разбор ключей через getopt (-t число потоков, -n количество случайных точек, -s seed, -f файл с точками), проверка аргументов через strtol, замер времени и вывод результата.
- show_threads.sh — запускает программу в фоне и показывает её потоки через ps -L, /proc/PID/status и /proc/PID/task.
- bench.sh — замеры времени для разного количества точек и потоков, считает ускорение и эффективность.

Используемые системные вызовы и функции POSIX:

- pthread_create() (clone3) — создание потока;
- pthread_join() (futex) — ожидание завершения потока;
- pthread_mutex_init(), pthread_mutex_destroy() — создание и удаление мьютекса;
- pthread_mutex_lock(), pthread_mutex_unlock() — захват и освобождение мьютекса (при ожидании внутри вызывается futex);
- gettid(), getpid() — получение TID потока и PID процесса для вывода;
- clock_gettime() — замер времени;
- malloc(), free() (mmap, brk) — выделение памяти;
- fopen(), fscanf(), fclose() (openat, read, close) — чтение файла с точками;
- write() (через printf) — вывод.

Функции pthread возвращают код ошибки, а не -1, и не выставляют errno, поэтому ошибки выводятся через strerror(err). Если pthread_create не смог создать поток, join делается только для уже созданных потоков, после чего освобождаются память и мьютексы.

Результаты

Сборка и запуск:

```
cmake -S . -B build
cmake --build build
./build/lab3 -t 4 -n 1000 -s 5
```

Вывод программы. Каждый поток печатает свой TID, порядок строк каждый раз разный, это показывает, что потоки работают параллельно и порядок их выполнения определяет планировщик:

```
main: PID=3044, points=1000, max threads=4
thread 1: TID=3045 started
thread 3: TID=3047 started
thread 2: TID=3046 started
thread 4: TID=3048 started
thread 4: TID=3048 finished, rows=254
thread 1: TID=3045 finished, rows=255
thread 3: TID=3047 finished, rows=249
thread 2: TID=3046 finished, rows=240
max triangle:
  A = #724 (-942.409, -941.716, -936.545)
  B = #844 (-770.151, 901.607, 954.749)
  C = #980 (928.864, 897.785, -925.869)
area: 2927970.404320
time: 48.666 ms
```

Количество потоков

Скрипт `show_threads.sh 4 3000` запускает программу в фоне с 4 потоками на 3000 точках и через секунду выводит информацию о потоках стандартными средствами Linux. У всех потоков один PID, а LWP (это TID) разные и совпадают с выводом программы. NLWP и строка `Threads` равны 5: четыре рабочих потока и главный. Главный поток спит в `pthread_join` (состояние `S`, 0% CPU), рабочие потоки выполняются (`R`) и загружают каждый своё ядро на 100%.

```
main: PID=3053, points=3000, max threads=4
thread 1: TID=3055 started
thread 2: TID=3056 started
thread 3: TID=3057 started
thread 4: TID=3058 started
----- ps -L -p 3053 -----
    PID     LWP NLWP %CPU STAT COMMAND
   3053    3053    5  0.0 Sl+  lab3
   3053    3055    5  100 Rl+  lab3
   3053    3056    5  100 Rl+  lab3
   3053    3057    5  100 Rl+  lab3
   3053    3058    5  100 Rl+  lab3
----- /proc/3053/status -----
Threads:  5
----- /proc/3053/task -----
3053  3055  3056  3057  3058
thread 1: TID=3055 finished, rows=753
thread 3: TID=3057 finished, rows=755
thread 4: TID=3058 finished, rows=747
thread 2: TID=3056 finished, rows=743
```

```
max triangle:
  A = #1237 (997.050, 954.455, 795.794)
  B = #1250 (-934.750, 923.191, -866.677)
  C = #1980 (-915.805, -964.360, 988.027)
area: 3005298.397822
time: 1285.164 ms
```

То же самое можно посмотреть командами `ps -eLf`, `top -H -p PID` или в `htop`.

Исследование ускорения и эффективности

Ускорение $S_p = T_1/T_p$, эффективность $E_p = S_p/p$, где T_p — время работы на p потоках. Замеры сделаны скриптом `bench.sh`: для каждой пары (n, p) три запуска, взято лучшее время. Процессор Intel Core Ultra 9 285H: 16 ядер без Hyper-Threading, из них 6 производительных (P), 8 энергоэффективных (E) и 2 low-power E-ядра. Ubuntu запущена в WSL2.

	$n = 200$		$n = 500$		$n = 1000$		$n = 2000$	
p	T, мс	S	T, мс	S	T, мс	S	T, мс	S
1	1.659	1.00	22.40	1.00	183.5	1.00	1555.7	1.00
2	0.810	2.05	11.36	1.97	95.9	1.91	800.1	1.94
4	0.616	2.69	6.67	3.36	47.7	3.84	433.1	3.59
8	0.573	2.90	3.81	5.88	27.1	6.78	246.8	6.30
12	0.591	2.81	3.43	6.54	20.6	8.92	181.5	8.57
16	0.701	2.37	2.89	7.76	18.0	10.22	171.2	9.09
24	0.692	2.40	3.43	6.54	18.2	10.07	175.9	8.84
32	0.849	1.95	3.11	7.21	18.1	10.16	176.7	8.80

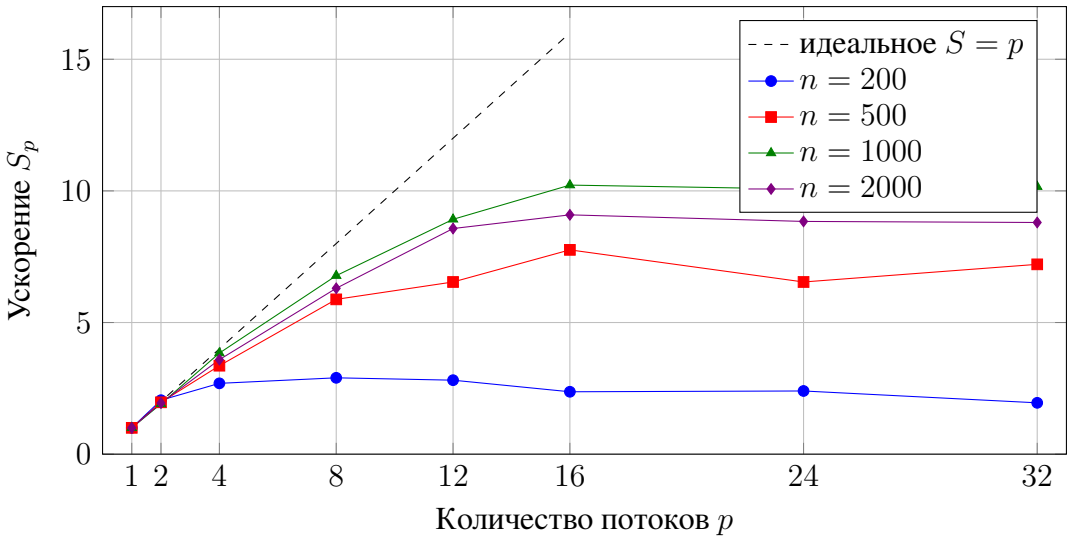


Рис. 1. Зависимость ускорения от количества потоков

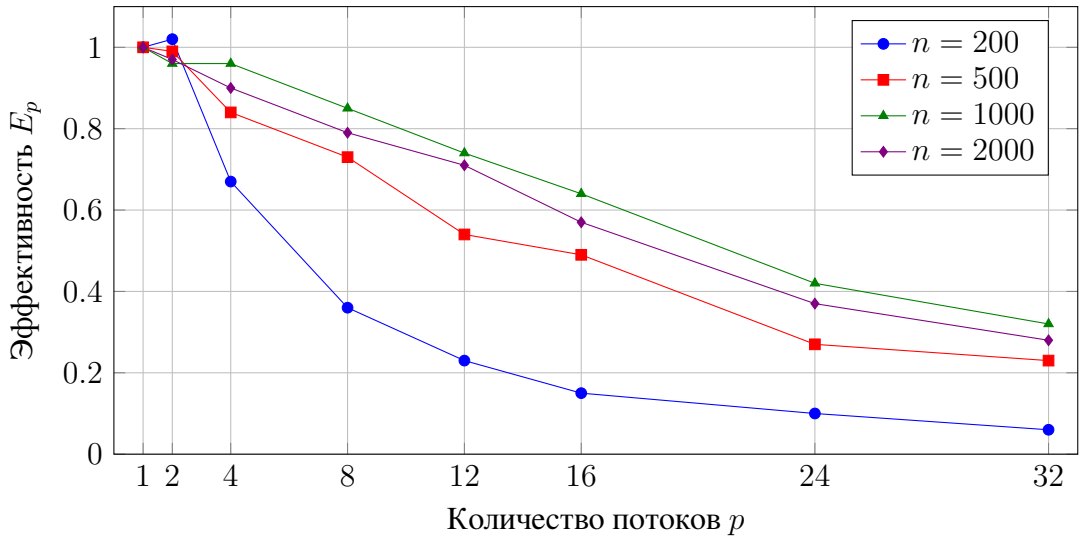


Рис. 2. Зависимость эффективности от количества потоков

Объяснение результатов.

1. При маленьких данных ($n = 200$, около 1.6 мс работы) ускорение быстро упирается примерно в 3 раза, а после 8 потоков время начинает расти. Создание каждого потока (выделение стека через `mmap`, `clone3`) и его join стоят десятки микросекунд, и при большом числе потоков эти накладные расходы сравнимы с самой работой. По закону Амдала $S_p = \frac{1}{f+(1-f)/p}$ ускорение ограничено долей f последовательной части, а здесь она большая.

2. С ростом n ускорение растёт: объём работы растёт как n^3, а накладные расходы на потоки от n почти не зависят, поэтому доля последовательной части уменьшается. На $n = 1000$ и $n = 2000$ эффективность до 4 потоков около 0.9–0.96, то есть почти идеальная. Потоки не обмениваются данными, массив точек только читается и помещается в кэш (2000 точек по 24 байта это 48 КБ), а мьютекс захватывается редко.

3. От 6 до 16 потоков ускорение продолжает расти, но эффективность падает до 0.6. Это связано с тем, что ядра у процессора разные: первые потоки попадают на быстрые P-ядра, следующие на более медленные E-ядра, последние на совсем слабые LP E-ядра. Кроме того, при нагрузке на все ядра ноутбук снижает частоту. В итоге максимум около 10 раз, а не 16.

4. Больше 16 потоков прироста нет: ядер всего 16, лишние потоки не работают параллельно, а делят ядра по времени, и добавляются переключения контекста. Ускорение остаётся на одном уровне, а эффективность падает примерно как $1/p$. Сильного замедления нет из-за динамической раздачи строк: поток, который реже получает процессор, просто берёт меньше строк.

5. На $n = 2000$ максимальное ускорение (9.1) немного меньше, чем на $n = 1000$ (10.2). Один запуск идёт больше секунды, процессор успевает нагреться и сбросить частоту. Также влияет то, что WSL2 работает в виртуальной машине, а Windows параллельно выполняет свои процессы. Значение $E > 1$ на $n = 200$, $p = 2$ это погрешность измерения очень маленького времени.

Выводы

После этой лабораторной работы я стал внимательнее продумывать, как разделить вычисления между потоками и какие общие данные нужно защищать мьютексом. В задаче с треугольником равное число строк перебора не означало равную нагрузку, поэтому работа выдавалась потокам по мере их освобождения. Замеры показали, что увеличение числа потоков не всегда уменьшает время: на небольших данных оно может расти из-за затрат на создание и согласование потоков.

Исходная программа

```h
#ifndef POINTS_H
#define POINTS_H

#include <stddef.h>

typedef struct {
    double x;
    double y;
    double z;
} Point;

Point *generate_points(size_t n, unsigned int seed);

Point *read_points(const char *filename, size_t *n);

double triangle_area(Point a, Point b, Point c);

#endif
```

Листинг 1: include/points.h

```h
#ifndef SEARCH_H
#define SEARCH_H

#include "points.h"

#include <stddef.h>

typedef struct {
    size_t i;
    size_t j;
    size_t k;
    double value;
} Triangle;

int find_max_triangle(const Point *points, size_t n, int threads, Triangle *result);

#endif
```

Листинг 2: include/search.h

```c
#include "points.h"

#include <math.h>
#include <stdio.h>
#include <stdlib.h>

Point *generate_points(size_t n, unsigned int seed) {
    Point *p = malloc(n * sizeof(Point));
    if (p == NULL) {
        perror("malloc");
        return NULL;
    }
```

```c
    srand(seed);
    for (size_t i = 0; i < n; i++) {
        p[i].x = (double)rand() / RAND_MAX * 2000.0 - 1000.0;
        p[i].y = (double)rand() / RAND_MAX * 2000.0 - 1000.0;
        p[i].z = (double)rand() / RAND_MAX * 2000.0 - 1000.0;
    }
    return p;
}

Point *read_points(const char *filename, size_t *n) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) {
        perror(filename);
        return NULL;
    }

    long count;
    if (fscanf(f, "%ld", &count) != 1 || count <= 0) {
        fprintf(stderr, "%s: bad number of points\n", filename);
        fclose(f);
        return NULL;
    }

    Point *p = malloc((size_t)count * sizeof(Point));
    if (p == NULL) {
        perror("malloc");
        fclose(f);
        return NULL;
    }

    for (long i = 0; i < count; i++) {
        if (fscanf(f, "%lf %lf %lf", &p[i].x, &p[i].y, &p[i].z) != 3) {
            fprintf(stderr, "%s: can't read point number %ld\n", filename, i + 1);
            free(p);
            fclose(f);
            return NULL;
        }
    }

    fclose(f);
    *n = (size_t)count;
    return p;
}

double triangle_area(Point a, Point b, Point c) {
    double ux = b.x - a.x;
    double uy = b.y - a.y;
    double uz = b.z - a.z;
    double vx = c.x - a.x;
    double vy = c.y - a.y;
    double vz = c.z - a.z;

    double cx = uy * vz - uz * vy;
    double cy = uz * vx - ux * vz;
    double cz = ux * vy - uy * vx;

    return sqrt(cx * cx + cy * cy + cz * cz) / 2.0;
```

```
}
```

Листинг 3: src/points.c

```c
#include "search.h"

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef struct {
    int num;
    long rows;
} ThreadArg;

static const Point *pts;
static size_t pts_count;
static size_t next_row;
static Triangle best;
static pthread_mutex_t row_mutex;
static pthread_mutex_t best_mutex;

static void lock(pthread_mutex_t *m) {
    int err = pthread_mutex_lock(m);
    if (err != 0) {
        fprintf(stderr, "pthread_mutex_lock: %s\n", strerror(err));
        exit(EXIT_FAILURE);
    }
}

static void unlock(pthread_mutex_t *m) {
    int err = pthread_mutex_unlock(m);
    if (err != 0) {
        fprintf(stderr, "pthread_mutex_unlock: %s\n", strerror(err));
        exit(EXIT_FAILURE);
    }
}

static void *worker(void *arg) {
    ThreadArg *a = arg;
    printf("thread %d: TID=%d started\n", a->num, gettid());

    Triangle local;
    local.i = 0;
    local.j = 0;
    local.k = 0;
    local.value = -1.0;

    while (1) {
        lock(&row_mutex);
        size_t i = next_row;
        next_row++;
        unlock(&row_mutex);

        if (i + 2 >= pts_count) {
```

```
            break;
        }

        Point A = pts[i];
        for (size_t j = i + 1; j < pts_count; j++) {
            double ux = pts[j].x - A.x;
            double uy = pts[j].y - A.y;
            double uz = pts[j].z - A.z;
            for (size_t k = j + 1; k < pts_count; k++) {
                double vx = pts[k].x - A.x;
                double vy = pts[k].y - A.y;
                double vz = pts[k].z - A.z;

                double cx = uy * vz - uz * vy;
                double cy = uz * vx - ux * vz;
                double cz = ux * vy - uy * vx;
                double s = cx * cx + cy * cy + cz * cz;

                if (s > local.value) {
                    local.value = s;
                    local.i = i;
                    local.j = j;
                    local.k = k;
                }
            }
        }
        a->rows++;
    }

    lock(&best_mutex);
    if (local.value > best.value || (local.value == best.value && local.i < best.i)) {
        best = local;
    }
    unlock(&best_mutex);

    printf("thread %d: TID=%d finished, rows=%ld\n", a->num, gettid(), a->rows);
    return NULL;
}

int find_max_triangle(const Point *points, size_t n, int threads, Triangle *result) {
    pts = points;
    pts_count = n;
    next_row = 0;
    best.i = 0;
    best.j = 0;
    best.k = 0;
    best.value = -1.0;

    int err = pthread_mutex_init(&row_mutex, NULL);
    if (err != 0) {
        fprintf(stderr, "pthread_mutex_init: %s\n", strerror(err));
        return -1;
    }
    err = pthread_mutex_init(&best_mutex, NULL);
    if (err != 0) {
        fprintf(stderr, "pthread_mutex_init: %s\n", strerror(err));
        pthread_mutex_destroy(&row_mutex);
        return -1;
```

```
    }

    pthread_t *ids = malloc((size_t)threads * sizeof(pthread_t));
    ThreadArg *args = malloc((size_t)threads * sizeof(ThreadArg));
    if (ids == NULL || args == NULL) {
        perror("malloc");
        free(ids);
        free(args);
        pthread_mutex_destroy(&row_mutex);
        pthread_mutex_destroy(&best_mutex);
        return -1;
    }

    int ret = 0;
    int created = 0;
    for (int t = 0; t < threads; t++) {
        args[t].num = t + 1;
        args[t].rows = 0;
        err = pthread_create(&ids[t], NULL, worker, &args[t]);
        if (err != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(err));
            ret = -1;
            break;
        }
        created++;
    }

    for (int t = 0; t < created; t++) {
        err = pthread_join(ids[t], NULL);
        if (err != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(err));
            ret = -1;
        }
    }

    free(ids);
    free(args);
    pthread_mutex_destroy(&row_mutex);
    pthread_mutex_destroy(&best_mutex);

    if (ret == 0) {
        *result = best;
    }
    return ret;
}
```

Листинг 4: src/search.c

```
#include "points.h"
#include "search.h"

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

```

```c
static void usage(const char *prog) {
    fprintf(stderr, "usage: %s -t threads (-n count [-s seed] | -f file)\n", prog);
    fprintf(stderr, " -t max number of threads\n");
    fprintf(stderr, " -n generate count random points\n");
    fprintf(stderr, " -s seed for random generator (default 1)\n");
    fprintf(stderr, " -f read points from file\n");
}

static long parse_number(const char *str, const char *name) {
    char *end;
    errno = 0;
    long value = strtol(str, &end, 10);
    if (errno != 0 || *end != '\0' || end == str || value <= 0) {
        fprintf(stderr, "bad value for %s: %s\n", name, str);
        exit(EXIT_FAILURE);
    }
    return value;
}

static void print_point(const char *name, size_t index, Point p) {
    printf(" %s = #%zu (%.3f, %.3f, %.3f)\n", name, index, p.x, p.y, p.z);
}

int main(int argc, char *argv[]) {
    int threads = 0;
    long count = 0;
    unsigned int seed = 1;
    char *filename = NULL;

    int opt;
    while ((opt = getopt(argc, argv, "t:n:s:f:")) != -1) {
        switch (opt) {
        case 't':
            threads = (int)parse_number(optarg, "-t");
            break;
        case 'n':
            count = parse_number(optarg, "-n");
            break;
        case 's':
            seed = (unsigned int)parse_number(optarg, "-s");
            break;
        case 'f':
            filename = optarg;
            break;
        default:
            usage(argv[0]);
            return EXIT_FAILURE;
        }
    }

    if (threads <= 0 || (count == 0 && filename == NULL) || (count != 0 && filename !=
         NULL)) {
        usage(argv[0]);
        return EXIT_FAILURE;
    }

    size_t n;
    Point *points;
```

```c
    if (filename != NULL) {
        points = read_points(filename, &n);
    } else {
        n = (size_t)count;
        points = generate_points(n, seed);
    }
    if (points == NULL) {
        return EXIT_FAILURE;
    }

    if (n < 3) {
        fprintf(stderr, "need at least 3 points, got %zu\n", n);
        free(points);
        return EXIT_FAILURE;
    }

    printf("main: PID=%d, points=%zu, max threads=%d\n", getpid(), n, threads);

    struct timespec start, end;
    if (clock_gettime(CLOCK_MONOTONIC, &start) == -1) {
        perror("clock_gettime");
        free(points);
        return EXIT_FAILURE;
    }

    Triangle res;
    if (find_max_triangle(points, n, threads, &res) != 0) {
        free(points);
        return EXIT_FAILURE;
    }

    if (clock_gettime(CLOCK_MONOTONIC, &end) == -1) {
        perror("clock_gettime");
        free(points);
        return EXIT_FAILURE;
    }

    double ms = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) /
         1000000.0;

    printf("max triangle:\n");
    print_point("A", res.i, points[res.i]);
    print_point("B", res.j, points[res.j]);
    print_point("C", res.k, points[res.k]);
    printf("area: %.6f\n", triangle_area(points[res.i], points[res.j], points[res.k]))
        ;
    printf("time: %.3f ms\n", ms);

    free(points);
    return EXIT_SUCCESS;
}
```

Листинг 5: src/main.c

```
cmake_minimum_required(VERSION 3.16)
project(os_lab3 LANGUAGES C)

```

```
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

add_compile_options(-Wall -Wextra -Wpedantic)
add_compile_definitions(_GNU_SOURCE)

find_package(Threads REQUIRED)

add_executable(lab3 src/main.c src/points.c src/search.c)
target_include_directories(lab3 PRIVATE include)
target_link_libraries(lab3 PRIVATE Threads::Threads m)
```

Листинг 6: CMakeLists.txt

```
#!/bin/bash

THREADS=${1:-4}
COUNT=${2:-3000}

./build/lab3 -t "$THREADS" -n "$COUNT" &
PID=$!
sleep 1

echo "----- ps -L -p $PID -----"
ps -L -o pid,lwp,nlwp,pcpu,stat,comm -p "$PID"

echo "----- /proc/$PID/status -----"
grep Threads /proc/"$PID"/status

echo "----- /proc/$PID/task -----"
ls /proc/"$PID"/task

wait "$PID"
```

Листинг 7: show_threads.sh

```
#!/bin/bash

SIZES="200 500 1000 2000"
THREADS="1 2 4 8 12 16 24 32"
RUNS=3

for n in $SIZES; do
    echo "n = $n"
    printf "%8s %12s %10s %12s\n" "threads" "time_ms" "speedup" "efficiency"
    t1=""
    for t in $THREADS; do
        best=""
```

```
        for r in $(seq $RUNS); do
            ms=$(./build/lab3 -t "$t" -n "$n" | grep "time:" | awk '{print $2}')
            if [ -z "$best" ] || [ "$(echo "$ms < $best" | bc)" -eq 1 ]; then
                best=$ms
            fi
        done
        if [ -z "$t1" ]; then
            t1=$best
        fi
        s=$(echo "scale=3; $t1 / $best" | bc)
        e=$(echo "scale=3; $s / $t" | bc)
        printf "%8s %12s %10s %12s\n" "$t" "$best" "$s" "$e"
    done
    echo
done
```

Листинг 8: bench.sh

Strace

Запуск:

```
strace -f -e trace=mmap,mprotect,clone3,futex,write,exit,exit_group ./build/lab3 -t 2
    -n 50
```

Ниже приведена часть вывода начиная с создания первого потока (до этого идёт загрузка программы и библиотек).

Перед каждым `clone3` glibc выделяет стек потока через `mmap` (около 8 МБ с флагом `MAP_STACK`) и через `mprotect` делает его доступным, оставляя внизу защитную страницу. В `clone3` видны флаги `CLONE_VM`, `CLONE_FILES`, `CLONE_SIGHAND`, `CLONE_THREAD`: новый поток работает в той же памяти и с теми же дескрипторами, а результатом вызова является его TID. Затем главный поток вызывает `futex` с `FUTEX_WAIT` — это `pthread_join`, он спит, пока поток не завершится. Поток выводит свои сообщения (`write`) и завершается вызовом `exit`, после чего ядро будит главный поток. В конце главный поток выводит результат и завершает процесс через `exit_group`. Вызовы `pthread_mutex_lock` в strace не видны, потому что свободный мьютекс захватывается атомарной операцией без системного вызова.

```
3035 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0
    x703968dff000
3035 mprotect(0x703968e00000, 8388608, PROT_READ|PROT_WRITE) = 0
3035 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|
    CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0
    x7039695ff990, parent_tid=0x7039695ff990, exit_signal=0, stack=0x703968dff000,
    stack_size=0x7fff80, tls=0x7039695ff6c0} => {parent_tid=[3036]}, 88) = 3036
3035 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0
    x7039685fe000
3035 mprotect(0x7039685ff000, 8388608, PROT_READ|PROT_WRITE) = 0
3036 write(1, "thread 1: TID=3036 started\n", 27 <unfinished ...>
3035 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|
    CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0
    x703968dfe990, parent_tid=0x703968dfe990, exit_signal=0, stack=0x7039685fe000,
    stack_size=0x7fff80, tls=0x703968dfe6c0} <unfinished ...>
3036 <... write resumed>) = 27
3035 <... clone3 resumed> => {parent_tid=[3037]}, 88) = 3037
```

```
3036 write(1, "thread 1: TID=3036 finished, row"..., 37 <unfinished ...>
3035 futex(0x7039695ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 3036, NULL,
    FUTEX_BITSET_MATCH_ANY <unfinished ...>
3036 <... write resumed>) = 37
3037 write(1, "thread 2: TID=3037 started\n", 27 <unfinished ...>
3036 exit(0 <unfinished ...>
3037 <... write resumed>) = 27
3036 <... exit resumed>) = ?
3036 +++ exited with 0 +++
3035 <... futex resumed>) = 0
3037 write(1, "thread 2: TID=3037 finished, row"..., 36 <unfinished ...>
3035 futex(0x703968dfe990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 3037, NULL,
    FUTEX_BITSET_MATCH_ANY <unfinished ...>
3037 <... write resumed>) = 36
3037 exit(0) = ?
3037 +++ exited with 0 +++
3035 <... futex resumed>) = 0
3035 write(1, "max triangle:\n", 14) = 14
3035 write(1, " A = #19 (-959.954, -84.597, -8"..., 40) = 40
3035 write(1, " B = #41 (520.950, 969.503, 870"..., 38) = 38
3035 write(1, " C = #47 (900.208, -894.941, 43"..., 38) = 38
3035 write(1, "area: 2191468.557596\n", 21) = 21
3035 write(1, "time: 2.453 ms\n", 15) = 15
3035 exit_group(0) = ?
3035 +++ exited with 0 +++
```